

--	--	--	--	--	--	--	--	--	--

Dataset D – Event Booking System

Each event object contains:

- eventId
- eventName
- location
- date
- ticketPrice
- ticketsAvailable
- ticketsSold
- status

Dataset contains **inconsistent and invalid entries**.
Students must handle them appropriately.

Application Requirements

You must build a React application using:

- React Context
- useReducer
- React Router

Mandatory Routes

/events
/events/:id
/filter
/stats

Mandatory Rules

- All data must be stored in **Context (Single Source of Truth)**
- All updates must go through **Reducer only**
- No hardcoding dataset
- No storing derived values in state
- Must use **map, filter, reduce**

Testing Requirements (Mandatory for Evaluation)

1. Use test IDs:

data-testid="total-events"
data-testid="active-events"
data-testid="soldout-events"
data-testid="event-item"

2. Expose computed values:

```
window.appState = {
  totalEvents,
  activeEvents,
  soldOutEvents
};
```

Question 1 – Display Valid Events

Route: /events

Requirements:

Display only **valid events**

Valid Event Conditions:

- eventName must exist and not empty
- ticketPrice > 0
- ticketsAvailable ≥ 0
- ticketsSold ≥ 0
- ticketsSold ≤ ticketsAvailable
- status must be one of: "Active", "Sold Out", "Cancelled"

Constraints:

- Must use .filter()
- Must use Context data
- Must not mutate dataset

Edge Cases:

- Missing eventName → display "Unnamed Event"
- Missing date → display "Date Not Available"

Question 2 – Event Detail View

Route: /events/:id

Requirements:

Display full event details

Calculate:

remainingTickets = ticketsAvailable - ticketsSold

Constraints:

- Must compute dynamically
- Must validate ID

Edge Cases:

- Invalid ID → show “Event not found”
- Negative result → handle safely

Question 3 – Filter Events by Location

Route: /filter

Requirements:

Filter events by **location**

Constraints:

- Must use .filter()
- Must be case-insensitive
- Must not refetch data

Validation:

- Empty input → show error
- Invalid location → show “No events found”

Interdependency:

- Must filter only **valid events (Q1 rules)**

Question 4 – Event Status Correction

Route: /events

Requirements:

Implement logic to **correct event status automatically**

Business Rules (Critical):

- If ticketsSold === ticketsAvailable
→ status must be "Sold Out"
- If status is "Cancelled"
→ it must NOT change

Constraints:

- Must use reducer
- Must not directly mutate state

Edge Cases:

- Invalid event → ignore
- Already correct status → no duplicate update

Interdependency:

Changes must reflect in:

- Event list
- Stats page

Question 5 – Events Analytics Dashboard

Route: /stats

Requirements:

Display:

- Total valid events
- Active events
- Sold Out events

Constraints:

- Must use .reduce()
- Must not store values in state
- Must compute dynamically

Edge Cases:

- Ignore invalid events
- Invalid status → exclude

Expected Output:

```
{  
  "totalEvents": 30,  
  "activeEvents": 18,  
  "soldOutEvents": 8  
}
```

Testing Requirement:

```
<div data-testid="total-events">30</div>  
<div data-testid="active-events">18</div>  
<div data-testid="soldout-events">8</div>
```

Global State Requirement:

```
window.appState = {  
  totalEvents: 30,  
  activeEvents: 18,  
  soldOutEvents: 8  
};
```